UNIVERSITY OF PISA

Lesson 5

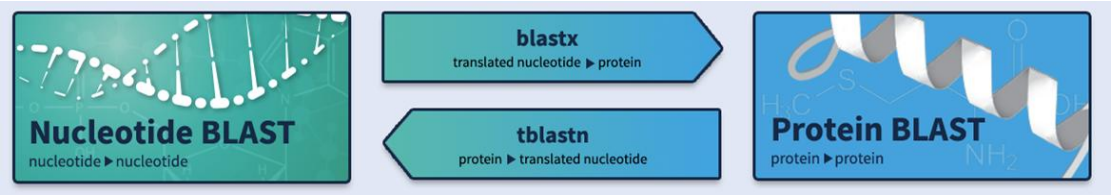
Silvia Giulia Galfrè

Bio.Blast

- https://biopython.org/docs/latest/Tutorial/chapter_blast.html
  Bio.Blast module
  - It can use Bio.Blast.NCBIWWW module for running over the internet
- Executing BLAST on the NCBI servers
  - send the request - no more than 1 every 10 seconds
  - wait – ask for results no more than once a minute
  - run heavy queries (>50 requests) over weekend or at night
  - include email address so you can be contacted
  - Use the optional email parameter so the NCBI can contact you if there is a problem.
    - explicitly set this as a parameter with each call
    - OR set global email address

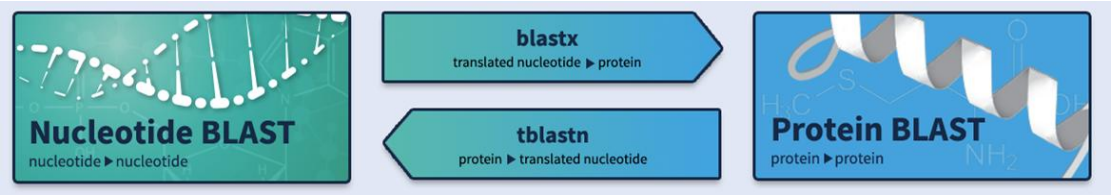
```
>>> from Bio import Entrez
>>> Entrez.email = "A.N.Other@example.com"
```

Bio.Blast



- Sending requests to NCBI: Bio.Blast.NCBIWWW
  - Function `qblast(program, database, sequence,...)`:
    - `program` - *blastn*, *blastp*, *blastx*, *tblastn* or *tblastx*
    - `database` - database to query - “nt”, “nr”, “swissprot”, (see documentation for complete list or web site menu)
    - `sequence` - query sequence, can be a GenBank identifier, or a string
    - ftp://ftp.ncbi.nlm.nih.gov/pub/factsheets/HowTo_BLASTGuide.pdf

Bio.Blast



- Function `qblast(program, database, sequence,...)`:
  - other options - similar to web tool
    - `format_type` - control format of result: "HTML", "Text", "ASN.1", or "XML".
    - `expect` - E-value threshold
  - returns a handle to read or parse

Xml output

- Hierarchically organized
- Stored also all input parameters


```xml
<?xml version="1.0" encoding="US-ASCII"?>
<!DOCTYPE BlastOutput PUBLIC "-//NCBI//NCBI BlastOutput/EN" "http://www.ncbi.nlm.nih.go
<BlastOutput>
  <BlastOutput_program>blastn</BlastOutput_program>
  <BlastOutput_version>BLASTN 2.17.0+</BlastOutput_version>
  <BlastOutput_reference>Stephen F. Altschul, Thomas L. Madden, Alejandro A. Sch&amp;au
  <BlastOutput_db>core_nt</BlastOutput_db>
  <BlastOutput_query-ID>Query_400003</BlastOutput_query-ID>
  <BlastOutput_query-def>My_seq</BlastOutput_query-def>
  <BlastOutput_query-len>1960</BlastOutput_query-len>
  <BlastOutput_param>
    <Parameters>
      <Parameters_expect>10</Parameters_expect>
      <Parameters_sc-match>2</Parameters_sc-match>
      <Parameters_sc-mismatch>-3</Parameters_sc-mismatch>
      <Parameters_gap-open>5</Parameters_gap-open>
      <Parameters_gap-extend>2</Parameters_gap-extend>
      <Parameters_filter>L;m;</Parameters_filter>
    </Parameters>
  </BlastOutput_param>
<BlastOutput_iterations>
<Iteration>
  <Iteration_iter-num>1</Iteration_iter-num>
  <Iteration_query-ID>Query_400003</Iteration_query-ID>
  <Iteration_query-def>My_seq</Iteration_query-def>
  <Iteration_query-len>1960</Iteration_query-len>
<Iteration_hits>
<Hit>
  <Hit_num>1</Hit_num>
  <Hit_id>gi|14589750|gb|AC016746.8|</Hit_id>
  <Hit_def>Homo sapiens BAC clone RP11-486F17 from 2, complete sequence</Hit_def>
  <Hit_accession>AC016746</Hit_accession>
  <Hit_len>186218</Hit_len>
  <Hit_hsps>
    <Hsp>
```

The BLAST Record class

- It stores the information provided by BLAST for a **single query**
- It is a list of Bio.Blast.Hit objects
- It has two attributes:
  - query: A SeqRecord object which may contain some or all of the following information:
    - query.id: SeqId of the query;
    - query.description: Definition line of the query;
    - query.seq: The query sequence.
  - stat: A dictionary with statistical data of the BLAST hit. You may find the following keys in this dictionary:
    - 'db-num': number of sequences in BLAST db (integer);
    - 'db-len': length of BLAST db (integer);
    - 'hsp-len': effective HSP (High Scoring Pair) length (integer);
    - 'eff-space': effective search space (float);
    - 'kappa': Karlin-Altschul parameter K (float);
    - 'lambda': Karlin-Altschul parameter Lambda (float);
    - 'entropy': Karlin-Altschul parameter H (float)

code

The BLAST hit class

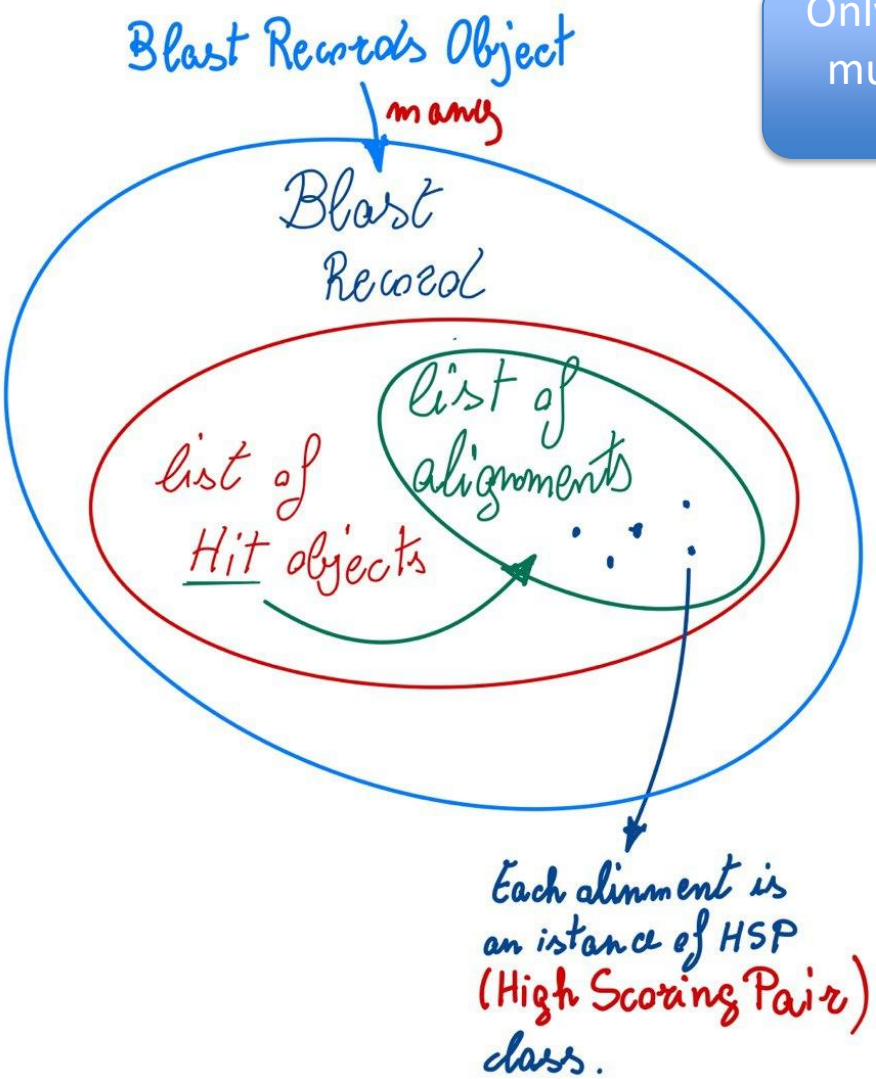
- Each Bio.Blast.Hit object in the blast_record list represents one BLAST hit of the query against a target.
- A hit consists of one or more alignments of the query against one target sequence.
- It is a subclass of Bio.Align.Alignments (plu ral),
- So it is a list of Bio.Align.Alignment (sing ular) objects.
- When aligning nucleotide sequences against the genome, the Hit object may consist of more than one Alignment
- For protein alignments, usually a hit consists of only one alignment

BLAST Records Class

- A single BLAST output file can contain output from multiple BLAST queries
- Stored in a Bio.Blast.Records object
- It is an iterator returning one Bio.Blast.Record object for each query.
- It has the attributes:

- `source` : The input data from which the `Bio.Blast.Records` object was constructed (this could be a file name or path, or a file-like object).
- `program` : The specific BLAST program that was used (e.g., 'blastn').
- `version` : The version of the BLAST program (e.g., 'BLASTN 2.2.27+').
- `reference` : The literature reference to the BLAST publication.
- `db` : The BLAST database against which the query was run (e.g., 'nr').
- `query` : A `SeqRecord` object which may contain some or all of the following information:
  - `query.id` : SeqId of the query;
  - `query.description` : Definition line of the query;
  - `query.seq` : The query sequence.
- `param` : A dictionary with the parameters used for the BLAST run. You may find the following keys in this dictionary:
  - `'matrix'` : the scoring matrix used in the BLAST run (e.g., 'BLOSUM62') (string);
  - `'expect'` : threshold on the expected number of chance matches (float);
  - `'include'` : e-value threshold for inclusion in multipass model in psiblast (float);
  - `'sc-match'` : score for matching nucleotides (integer);
  - `'sc-mismatch'` : score for mismatched nucleotides (integer;
  - `'gap-open'` : gap opening cost (integer);
  - `'gap-extend'` : gap extension cost (integer);
  - `'filter'` : filtering options applied in the BLAST run (string);
  - `'pattern'` : PHI-BLAST pattern (string);
  - `'entrez-query'` : Limit of request to Entrez query (string).

Only locally installed BLAST+ with a multi-FASTA query can produce a multi records xml



Homework

- Search UniProt for human protein APAF (O14727). Save it in FASTA format. Using BioPython do:
  - Load the file and check that the protein contains 1248 aa
  - Using BLASTP, search for sequences with high simirar to this sequence, in the “swissprot” database
  - Check which global parameters were used
  - List the alignment returned and alignment length
  - Repeat the search restricting the target sequences to the organism S. cerevisiae
  - Check the results from the last operation, listing the best alignments, and checking carefully in each the start position of the alignment in the query and in the sequence.
  - What do you conclude about the existence of homologous genes in the yeast for the human protein APAF?

MULTIPLE SEQUENCE ALIGNMENT

MSA: why?

- Because:
  - We can have multiple similar sequences form different organisms
  - In epidemiology we can have multiple sequeces form different strands
  - Etc.

| | | |

MSA

- It generalize the pairwise sequence alignment
- It can be local or global but usually we consider the **global** approach since the local is more related to motif search
- It is considered NP-hard in terms of complexity
- We still want an addictive objective function
- We still have a substitution matrix and a gap penalty

MSA objective function

- A common approach: sum of pairs (SP)

```
-RADNS
ARCD-A
AR-D-A
```

$$
\begin{aligned}
&Score \\
&= sm(A,A) + g) + 3 \cdot sm(R,R) + (sm(A,C) + g) + 3 \cdot sm(D,D) \\
&+ 2 \cdot g + 2 \cdot sm(S,A) + sm(A,A) \\
&= (4-8) + 3 \cdot 5 + (0-8) + 3 \cdot 6 + (-8 \cdot 2) + (1+1+4) = 11
\end{aligned}
$$

How can we approach MSA?

- Dynamic programming from PA?
- Yes… but it become exponential with as exponent the number or sequences
- Then in PA we fill a matrix…here we need a hypercube (d = numbe

Too complex!

Heuristic algorithms

- DP not usable in the normal biological situation since we always have many sequences!
- So, we not assure an optimal solution, but we can cope with the problem with acceptable complexity
- The heuristic algorithms for MSA can be classified as:
  - Progressive : first align 2 sq and then iteratively add the remaining
  - Iterative: consider an initial alignment and then try to improve it by moving, adding or removing gaps
  - Hybrid: can combine different strategies and use complementary information (e.g. proteins structure, libraries of good alignments)

1. Progressive- CLUSTAL

- **Big idea:** build the MSA step-by-step following a guide tree. Early decisions "freeze in" and influence everything that follows.
- **Fast pairwise distances**
  - CLUSTAL first estimates how similar every pair of sequences is.
  - It uses a **fast heuristic** algorithm to finds short word matches, extends them, and derives an approximate score/percent identity.
  - Result: a **distance or a similarity matrix**.

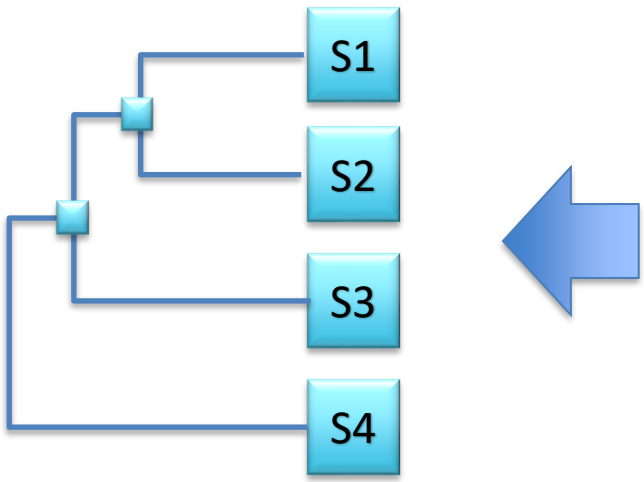
```
S1:PEEMSVSLDLTGGLPEATT…
S2:PEEMSVSLDLTGGLPEAST…
S3:SEELAAATDLTGAPSPAAA…
S4:PGPGPLAEVRDLPGSTSKE…
```

	S1	S2	S3	S4
S1	-		Similarity matrix	
S2	76	-		
S3	45	53	-	
S4	31	27	43	-

1. Progressive- CLUSTAL

- **Build a guide tree**
  - From the distance matrix, a **guide tree** is constructed (historically **UPGMA** or **Neighbor-Joining**).
  - This tree determines the **order** in which sequences (and later, profiles) will be aligned: closest relatives first, then progressively more distant ones.

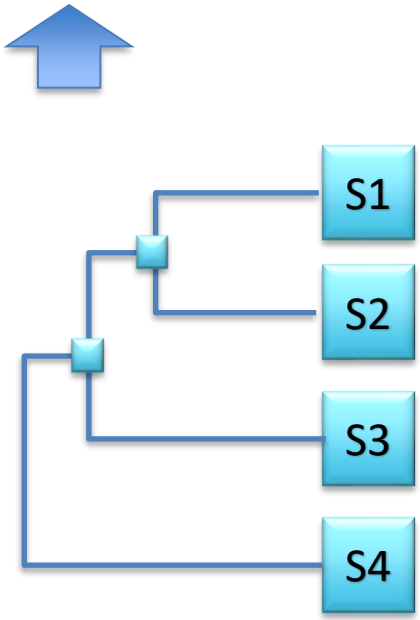
Guide tree



	S1	S2	S3	S4
S1	-		Similarity matrix	
S2	76	-		
S3	45	53	-	
S4	31	27	43	-

1. Progressive- CLUSTAL

```
S1:PEEMSVTS-LDLTGGLPEATT…
S2:PEEMSVAS-LDLTGGLPEAST…
S3:SEELAAATALDLG----APSP…
S4:PGPGPLAEVRDLPG-----ST…
```



- **Progressive alignment along the tree**
  - Start at the leaves (most similar sequences): perform a **global alignment** (Needleman–Wunsch) to align the closest pair.
  - Replace that pair with a **profile** (a position-specific summary that takes into accounts frequencies).
  - Move up the tree, aligning **sequence vs. profile** or **profile vs. profile** using DP
  - When profiles are aligned, columns are merged; gaps introduced earlier are **not re-optimized**

1. Progressive- CLUSTAL

```
S1:PEEMSVTS-LDLTGGLPEATT…
S2:PEEMSVAS-LDLTGGLPEAST…
S3:SEELAAATALDLG----APSP…
```

```
S4:PGPGPLAEVRDLPG-----ST…
```

- Adding a sequence to a profile: the score
- All possible character appearing in the profile , weighted for their frequencies

$$\text{Score} = (\text{sm}(\text{V},\text{L}) \cdot 2 + \text{sm}(\text{A},\text{L}))/3$$

1.1 Progressive-CLUSTALW

- More robust and biologically sensible:
  - **Sequence weighting**: down-weights over-represented, closely related sequences so they don't dominate the scoring.
  - **Position-/residue-specific gap penalties**:
    - Different penalties near termini vs. core.
    - Lower gap penalties in hydrophilic/low-complexity regions; higher in conserved motifs/secondary-structure-like regions.
    - Different gap costs for different residue types (e.g., discouraging gaps inside helices/beta strands).
  - **Choice of substitution matrices** by sequence divergence (e.g., BLOSUM/PAM scaled to similarity).
  - **Optional iteration** on subtrees to clean up local problems.
- These tricks reduce—but don't eliminate—the "early error propagation" inherent to progressive methods.
- It is not used anymore

1.2 Progressive-CLUSTAL **Omega**

What CLUSTAL Ω changed: It keeps the **progressive** spirit but scales to very large datasets and improves accuracy:

- **Fast k-mer–based distances + mBed embedding** to avoid computing all N^2 pairwise distances explicitly.
  - Count how many times each k-mer appears in each sequence → a k-mer count vector.
  - Compare two sequences by comparing their vectors (e.g., cosine/Jaccard/correlation or a simple shared-k-mer score): a quick approximation of similarity without doing full DP.
  - It's linear in sequence length and very fast in practice.
  - Why mBed "embedding" is needed: If you have N sequences, a classic guide tree needs all pairwise distances → about N(N-1)/2 comparisons (quadratic).
  - Even with fast k-mer distances, that's huge when N is in the tens/hundreds of thousands.
  - mBed avoids doing all pairs by turning each sequence into a short coordinate vector so that we can estimate distances indirectly (Euclidian distance in the embedding space).
- **Guide tree** built efficiently from the embedded distances (UPGMA/NJ-style clustering).

1.2 Progressive-CLUSTAL **Omega**

- **Profile-HMM / profile–profile alignment** (via HMM techniques under the hood) during the progressive phase, which is typically more sensitive and accurate for remote homology than raw profile scores.
- Designed for **speed and scale** (hundreds of thousands of sequences) with strong accuracy.

Progressive approach problem

- Wrong decision in early steps will not be corrected in the next steps influencing the whole alignment
- The worst results will be when sequences have low similarity

2. Iterative algorithms

- Start with an initial MSA (often progressive), then **repeatedly modify and re-align parts of it**, keeping only changes that **improve an objective** (SP score, expected accuracy, or a consistency score). This lets the method **undo early mistakes** and converge to a better alignment.

2. Iterative algorithms: core refinement patterns

1. **Tree-edge (profile–profile) refinement**
   - Build a guide tree from the current MSA.
   - Pick an edge → split the MSA into two profiles A|B.
   - Realign A vs B with DP
   - **Accept** if score improves. Repeat over many edges / rounds.
   - *Seen as a pattern in* **MUSCLE Stage-3** (SP score), and in some chases for **MAFFT**.

2. Iterative algorithms: core refinement patterns

2. **Posterior/MEA refinement (no consistency)**
   - Compute pairwise posterior match probabilities with a pair-HMM;
   - realign blocks/columns to **maximize expected accuracy**; accept if improved.

Less common as a stand-alone.

2. Iterative algorithms: core refinement patterns

3. **Stochastic/global re-optimization**
   - Random perturbations,
   - simulated annealing,
   - genetic algorithms;
   - repeatedly re-align altered regions and accept if the global objective improves.
   - *Examples:* classic GA/annealing MSAs (older literature).

3. Hybrid: core refinement patterns

- **Consistency-based + iteration**
  - Build a library of pairwise constraints (from pairwise alignments or posteriors).
  - Score MSAs by how **consistent** they are with the library.
  - Iteratively realign subprofiles to **maximize consistency**.
  - *Examples:* **T-Coffee**, **Expresso**, **M-Coffee** (and variants).

What is a pairwise constraints ?

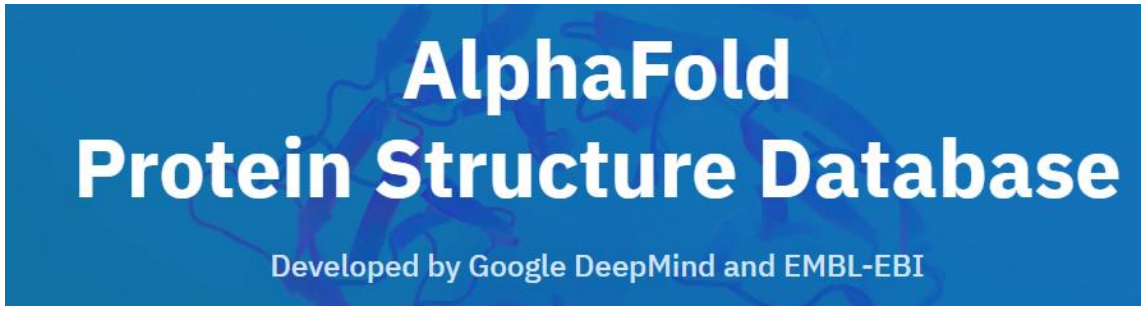
- A **constraint** is simply a claim like:
  - "Residue *i* of sequence X should align with residue *j* of sequence Y."
- We store that pair (X,i) $\leftrightarrow$ (Y,j) with a **weight** (confidence).
- The **library** is just the **union of all such weighted residue-pair constraints** gathered from every sequence pair

3. Hybrid: core refinement patterns

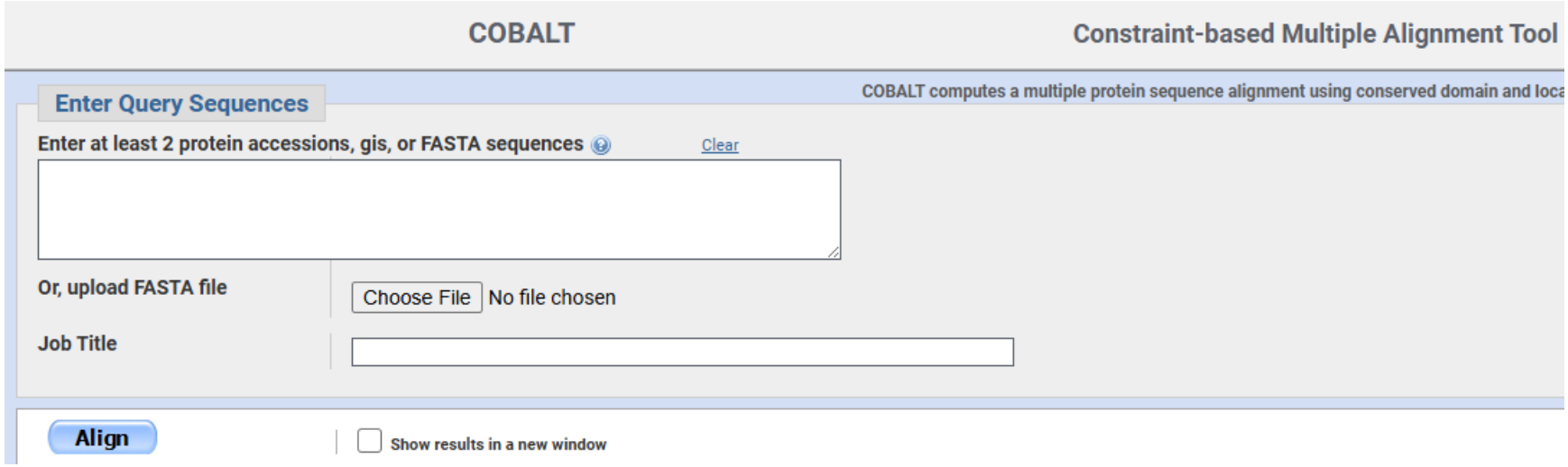
- **Posterior-probability/MEA refinement**
  - Compute pairwise **posterior match probabilities** (Pair-HMMs).
  - Iteratively realign blocks to **maximize expected accuracy (MEA)**, often focusing on low-confidence regions.
  - *ProbCons, MSAProbs, MAFFT L/E/G-INS-i use variants.*

MSA and AlphaFold

- MSA is a crucial input for AlphaFold, a deep learning system that predicts protein structures.
- AlphaFold uses the MSA to identify co-evolving amino acid pairs, which helps it infer the spatial proximity of residues and accurately predict the protein's 3D shape.



Online tools



https://www.ncbi.nlm.nih.gov/tools/cobalt/cobalt.cgi?CMD=Web

https://www.ebi.ac.uk/jdispatcher/msa

Explore Sequence Analysis Tools with

Job Dispatcher

European Bioinformatics Institute (EMBL-EBI)

Home Help & Privacy Recent Jobs

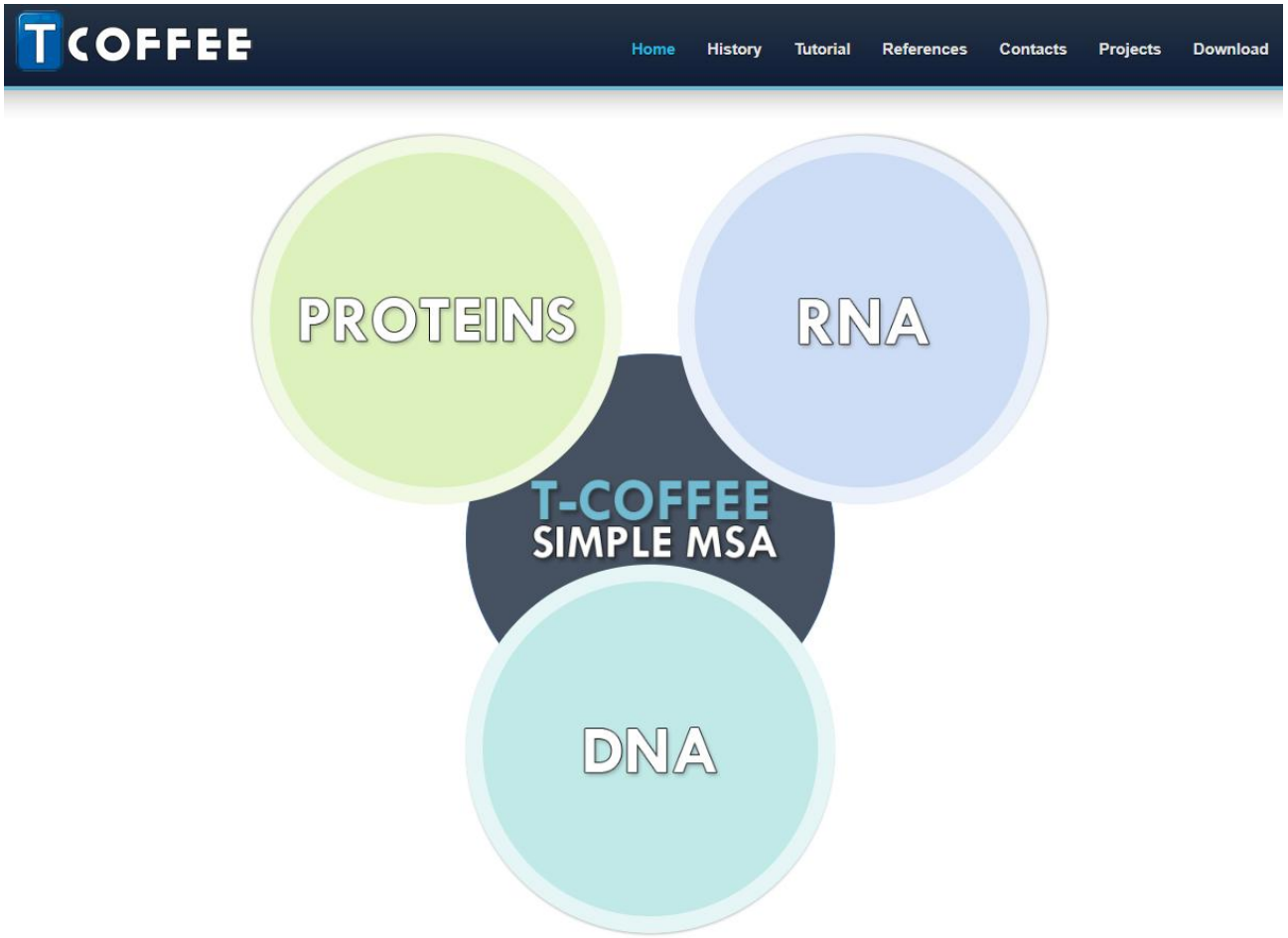
Feedback

Welcome to the Job Dispatcher website! If you need assistance or have feedback, please contact us.

Multiple Sequence Alignment

Multiple Sequence Alignment (MSA) is generally the alignment of three or more biological sequences (protein or nucleic acid) of similar length. From the output, homology can be inferred and the evolutionary relationships between the sequences studied. By contrast, Pairwise Sequence Alignment tools are used to identify regions of similarity that may indicate functional, structural and/or evolutionary relationships between two biological sequences.

https://tcoffee.crg.eu/apps/tcoffee/index.html



MSA in BioPython

Multiple Sequence Alignment objects

This chapter describes the older `MultipleSeqAlignment` class and the parsers in `Bio.AlignIO` that parse the output of sequence alignment software, generating `MultipleSeqAlignment` objects. By

Multiple Sequence Alignments we mean a collection of multiple sequences which have been aligned together – usually with the insertion of gap characters, and addition of leading or trailing gaps – such that all the sequence strings are the same length. Such an alignment can be regarded as a matrix of letters, where each row is held as a `SeqRecord` object internally.

We will introduce the `MultipleSeqAlignment` object which holds this kind of data, and the `Bio.AlignIO` module for reading and writing them as various file formats (following the design of the `Bio.SeqIO` module from the previous chapter). Note that both `Bio.SeqIO` and `Bio.AlignIO` can read and write sequence alignment files. The appropriate choice will depend largely on what you want to do with the data.

https://biopython.org/docs/latest/Tutorial/chapter_msa.html#